



COMPILADORES

TEMA 2. ANÁLISIS LÉXICO

Presenta: Mtro. David Martínez Torres

Universidad Tecnológica de la Mixteca

Instituto de Computación

Oficina No. 37

dtorres@gs.utm.mx



Contenido

1. Funcionalidad del analizador léxico
2. Especificación y reconocimiento de los símbolos de un lenguaje
3. Diseño e implementación de un analizador léxico
4. Control de errores léxicos

Estructura del programa Fuente

PREPROCESADOR

Programa Fuente

COMPILADOR

Analizador
Léxico

Analizador
sintáctico

Analizador
semántico

Tabla de
símbolo

Manejador
de Errores

Generador de
código intermedio

Optimizador
de código

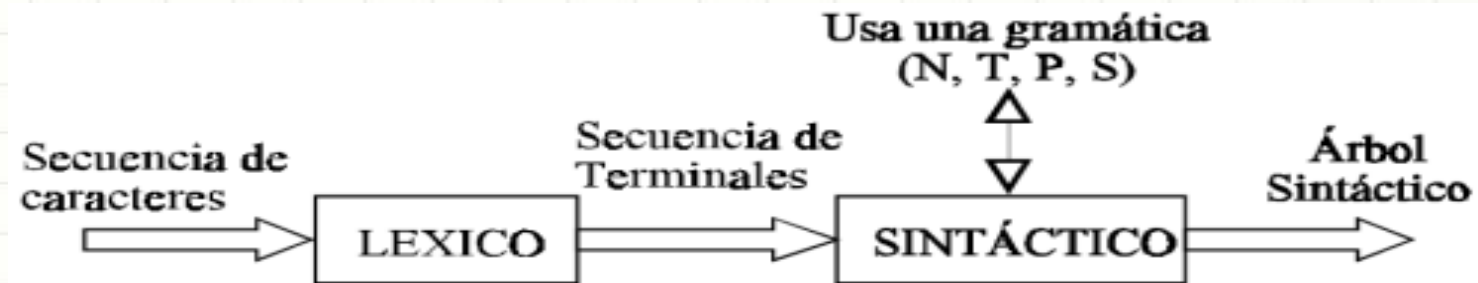
Generador
de código

Programa Objeto

Recordemos la estructura completa de un Compilador.

1. Funcionalidad del analizador léxico

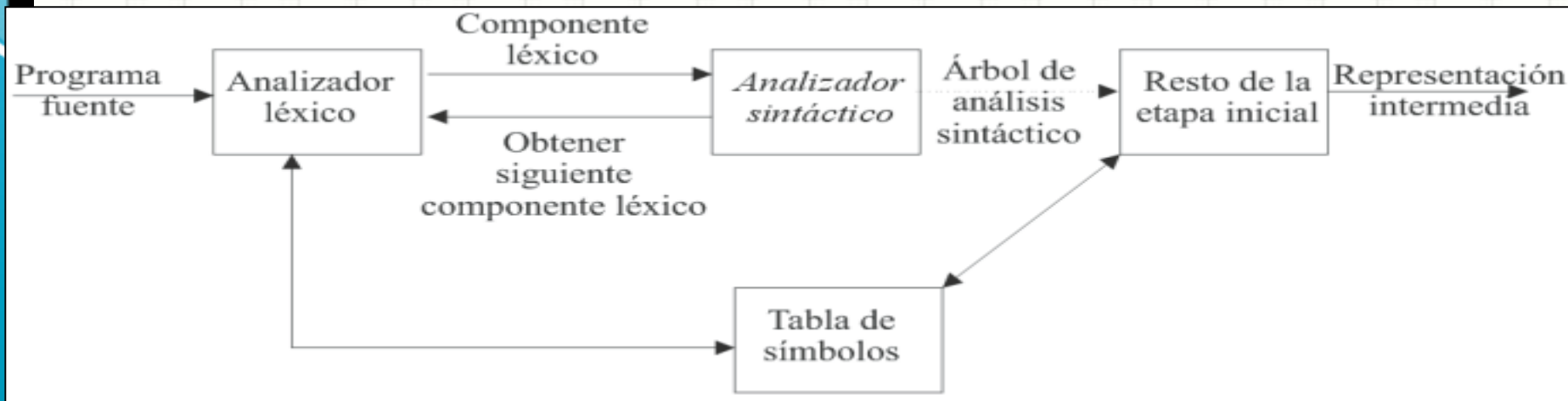
El **analizador léxico** es un scanner, recorre el programa de izquierda a derecha, símbolo a símbolo formando cadenas que si son reconocidas por un autómatas(reconoce expresiones regulares) serán componentes léxicos o tokens.



En caso de encontrar un símbolo no definido en el alfabeto, se enviará tal símbolo a una tabla de errores.

1. Funcionalidad del analizador léxico

El **analizador léxico** es la primera fase de un compilador. La secuencia de componentes léxicos que genera, los utiliza el analizador sintáctico para hacer el análisis. En la sig. figura, se muestra otra explicación.



1. Funcionalidad del analizador léxico

Salidas de un analizador léxico

1. Tira de tokens

- Se listan todos los componentes léxicos del programa fuente

2. Tabla de símbolos

- Contiene solo identificadores y campos como valor, dirección de memoria, función en la que se encuentra.

3. Mensajes de errores y tratamiento de errores

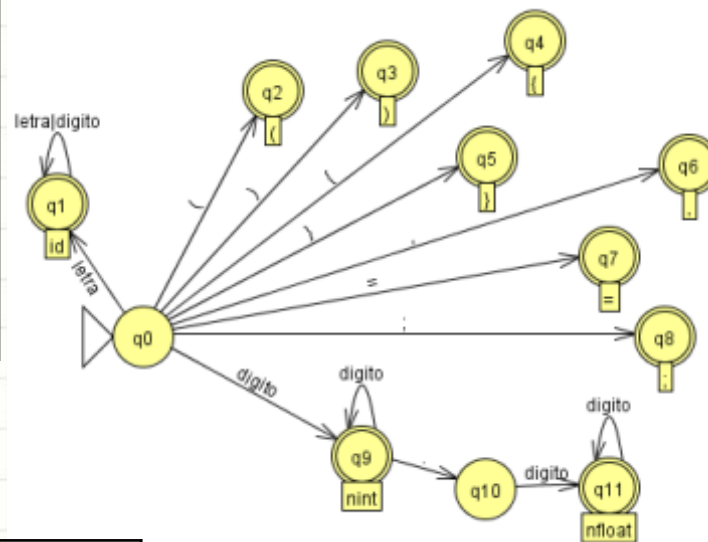
- Por ejemplo, si @ no pertenece al lenguaje, avisar de un error.

# línea	lexema	token	token (Recomendable)
1	int	pReservada	int
1	num	id	id
1	,	coma	,
1	suma	id	id
1	=	opAsig	=
1	0	numEntero	nint
1	;	pYComa	;

Programa fuente

```
int main() {  
int a, b=1 @;  
  
a=b+2;  
}
```

AFD



1. Tira de tokens

# línea	Lexema	Token
1	int	int
1	main	main
1	(	(
1	)	)
1	{	{
2	int	int
2	a	id."a"
2	,	,
2	b	id."b"
2	=	=
2	1	nint.1
2	;	;
4	a	id."a"
4	=	=
4	b	id."a"
4	+	+
4	2	nint.2
4	;	;
5	}	}

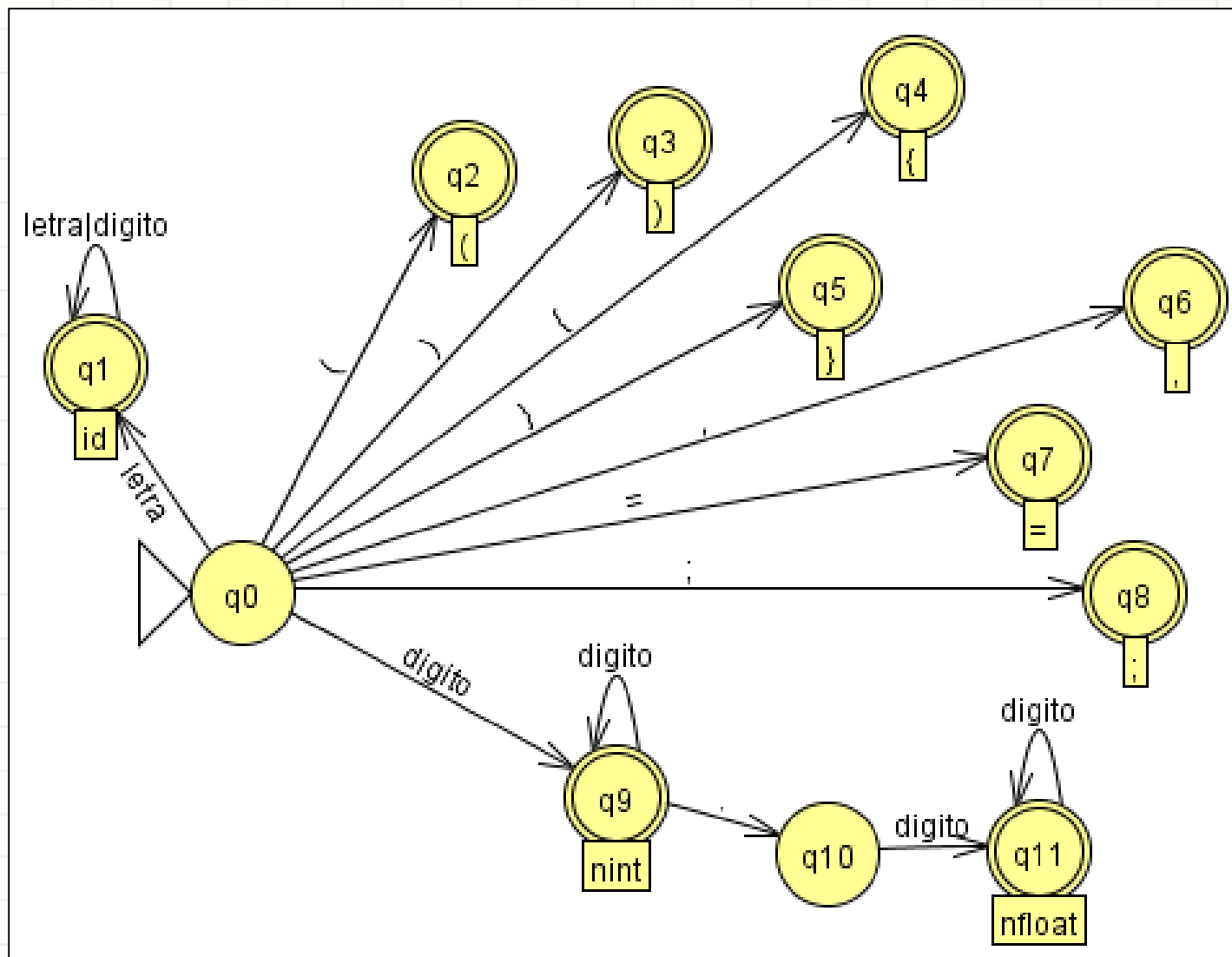
2. Tabla de símbolos

id	valor	tipo	función
a			
b			

3. Tabla de errores

# línea	Descripción
	En función 'main'
2	Error: @ símbolo no definido.

Bosquejo de un AFD para el análisis léxico



1. Funcionalidad del analizador léxico

Otras funcionalidades:

- Descarta los comentarios del programa.
- Ignora espacios en blanco, tabuladores, retorno de carro, etc, y en general, todo aquello que carezca de significado según la sintaxis del lenguaje.
- Llevar la cuenta del número de línea por la que va leyendo, por si se produce algún error, dar información sobre donde se ha producido.
- Puede hacer funciones de preprocesador.



2. Especificación y reconocimiento de los símbolos de un lenguaje



- Autómatas finitos
 - Reconocedor de cadenas de símbolos.
 - Expresiones regulares
- Autómata Finito No Determinista
- Autómata Finito Determinista

Autómatas Finitos No Deterministas AFN

- Permite transiciones épsilon ϵ
- Desde un estado con una entrada, puede tener más de una transición.
- Elementos de un AFN = (S, Σ, f, s_0, F)
 - S conjunto de estado.
 - Σ define el conjunto de símbolos de entrada.
 - f función de transición (estado, símbolo).
 - s_0 estado inicial.
 - F conjunto de estados finales o de aceptación.



Convertir una expresión regular a un AFN (Algoritmo de Thompson)

- Entrada: Una expresión regular r , con un alfabeto Σ
- Salida: Un AFN N que acepte $L(r)$
- Método:
 - Construir los sub AFN's a partir de las subexpresiones regulares mediante las siguientes reglas:

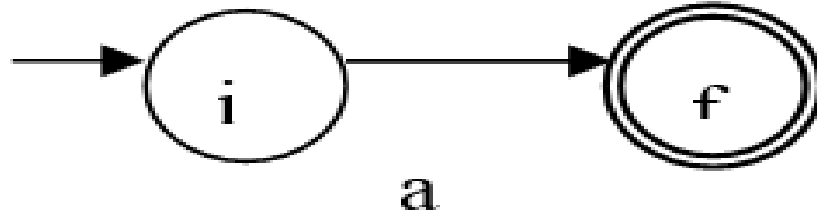


Convertir una expresión regular a un AFN (Algoritmo de Thompson)

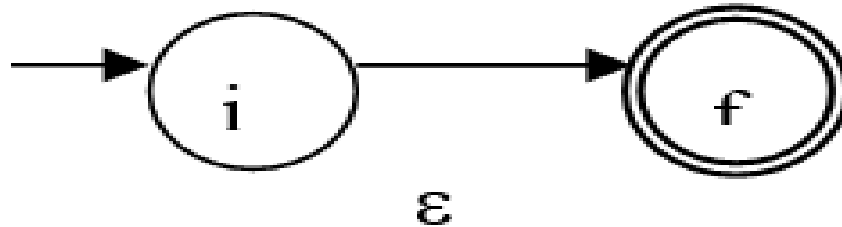
Método:

Construir los sub AFN's a partir de las subexpresiones regulares mediante las siguientes reglas:

1. a

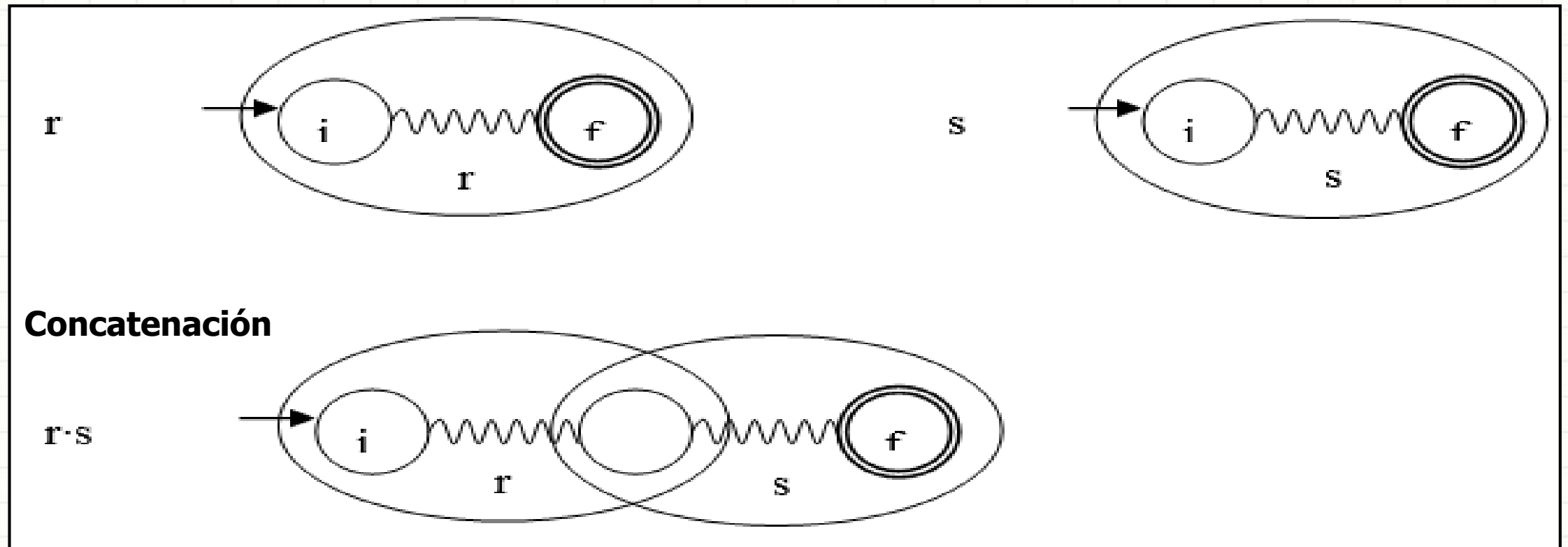


2. ϵ

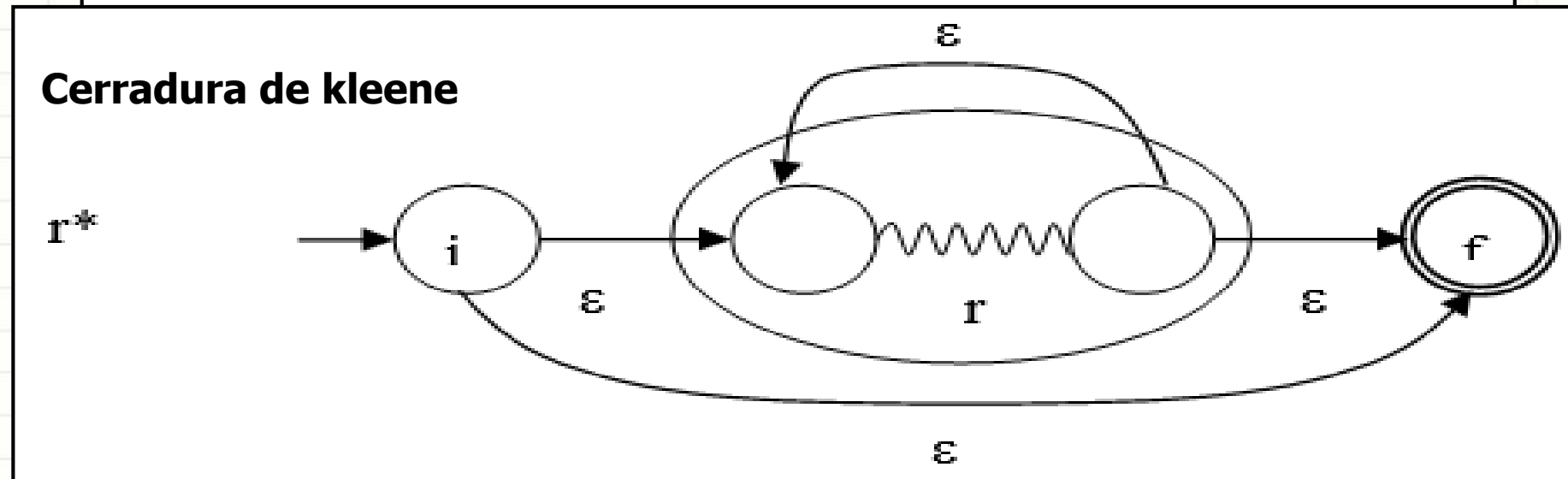
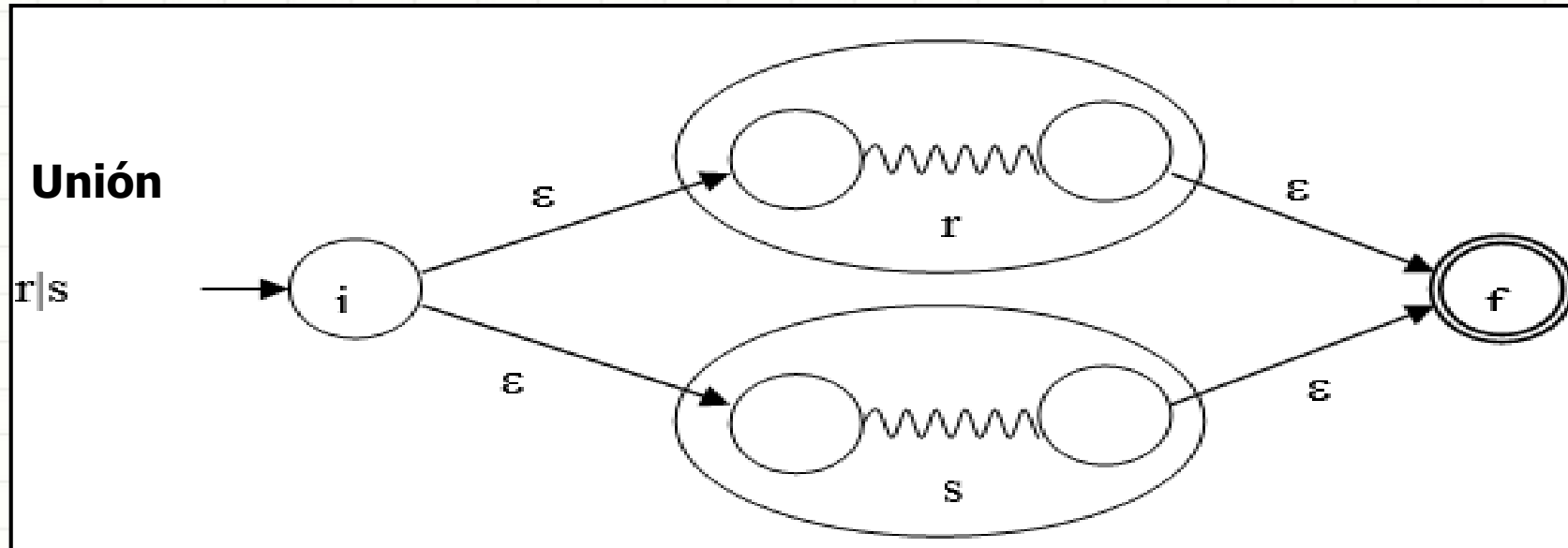


Convertir una expresión regular a un AFN (Algoritmo de Thompson)

- 3. Si r y s son expresiones regulares con los AFN's

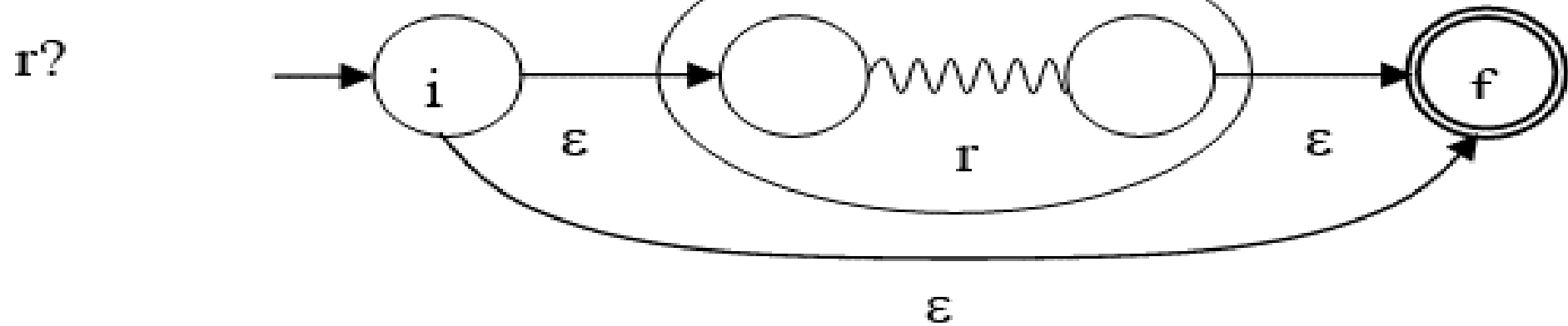


Convertir una expresión regular a un AFN (Algoritmo de Thompson)

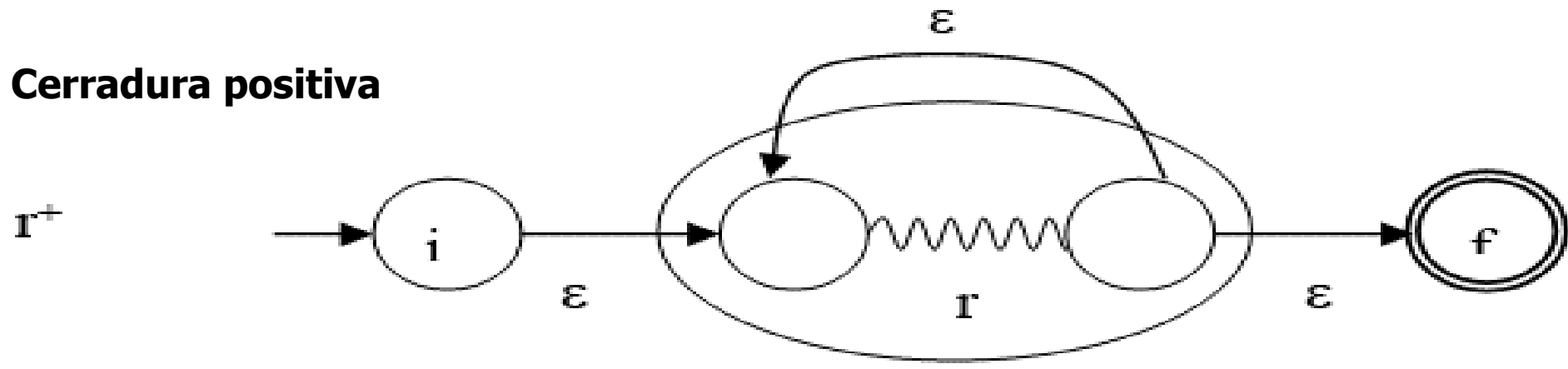


Convertir una expresión regular a un AFN (Algoritmo de Thompson)

Cerradura opcional



Cerradura positiva

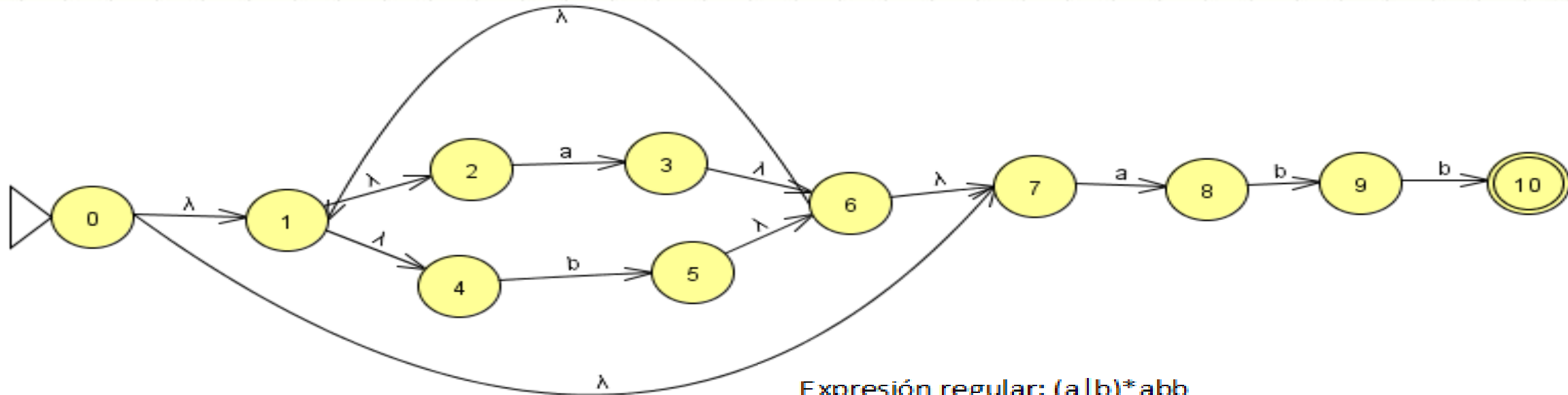


Convertir una expresión regular a un AFN (Algoritmo de Thompson)

Las operaciones anteriores son asociativas por la izquierda y tienen el siguiente orden de prioridad:

1. Cerraduras
2. Concatenación
3. Unión

De acuerdo a estas convenciones la siguiente expresión regular $(a) | ((b)^*(c))$, es equivalente a: $a | b^*c$



Ejemplo: Convertir la siguiente
expresión regular a un AFN:
 $(a|b)^*abb$

$\Sigma = \{a, b\}$

Expresión regular: $(a|b)^*abb$

$S = \{0, 1, 2, \dots, 10\}$

$s_0 = 0$

$\Sigma = a, b$

$F = 10$

Tabla de transiciones AFN

Edo	a	b	ϵ
0	-	-	{1, 7}
1	-	-	{2, 4}
2	3	-	-
3	-	-	6
4	-	5	-
5	-	-	6
6	-	-	{1, 7}
7	8	-	-
8	-	9	-
9	-	10	-
10	-	-	-

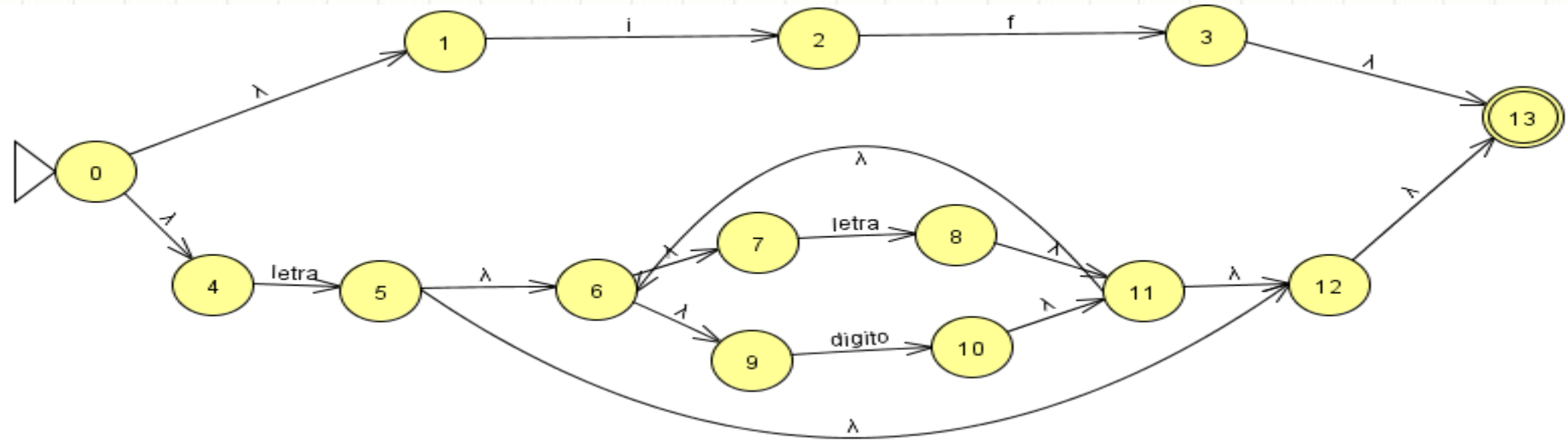
Convertir una expresión regular a un AFN (Algoritmo de Thompson)

Ejemplo: Convertir la siguiente expresión regular a un AFN:
`if | letra(letra | digito)*`

Donde:

`letra` $\rightarrow$ `a-zA-Z`

`Digito` $\rightarrow$ `0-9`



Ejemplo: Convertir la siguiente expresión regular a un AFN:

$i f | letra(letra | digito)^*$

Donde:

$letra \rightarrow a-zA-Z$

$Digito \rightarrow 0-9$

Tabla de transiciones de AFN $i f | letra(letra | digito)^*$

$S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13\}$

$\Sigma = i, f, a-zA-Z, 0-9$

$s_0 = 0$

$F = 13$

Edo	i	f	letra	digito	ϵ
0	-	-	-	-	{1,4}
1	2	-	-	-	-
2	-	3	-	-	-
3	-	-	-	-	13
4	-	-	5	-	-
5	-	-	-	-	{6,12}
6	-	-	-	-	{7,9}
7	-	-	8	-	-
8	-	-	-	-	11
9	-	-	-	10	-
10	-	-	-	-	11
11	-	-	-	-	{6,12}
12	-	-	-	-	13
13	-	-	-	-	-

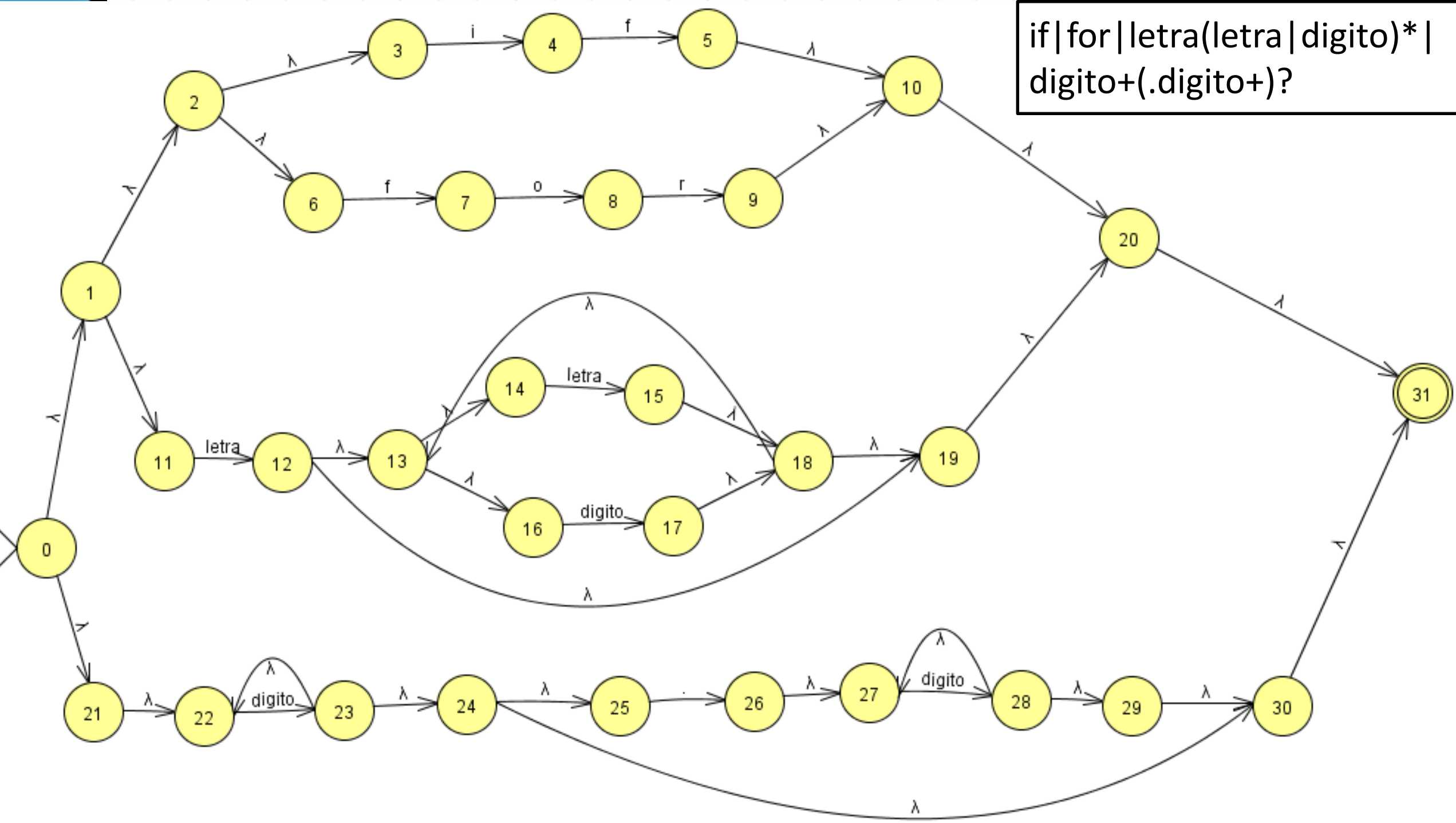
Convertir una expresión regular a un AFN (Algoritmo de Thompson)

Ejercicios: Convierta las siguientes expresiones regulares a su correspondiente AFN. Obtenga como salida su grafo y su tabla de transiciones.

- $a^*(bc^+)|(d|a)b?$
- $\text{letra}(\text{letra}|\text{digito})^*$
- $\text{if}|\text{letra}(\text{letra}|\text{digito})^*$
- $\text{digito}^+(\text{.digito}^+)?$
- $\text{if}|\text{for}|\text{letra}(\text{letra}|\text{digito})^*|\text{digito}^+(\text{.digito}^+)?$



if | for | letra(letra | digito)* |
digito+(.digito+)?



Autómata Finito Determinista

Los AFD son mucho más eficientes que los AFN en el reconocimiento de patrones, de ahí que se utilicen en la construcción de compiladores.

Sin embargo, son más difíciles de construir, debido a las siguientes restricciones:

- Ningún estado tiene transiciones ϵ
- Solo puede haber una transición de un estado con una entrada.



Convertir un AFN a un AFD (Algoritmo Construcción de conjuntos)

- **Entrada:** Un AFN N , como su grafo o en este caso su tabla de transiciones que obtuvimos del Algoritmo de Thompson.
- **Salida:** Un AFD D (tabla de transiciones)
- **Método:** El algoritmo construye una tabla de transiciones **tranD** para D . Cada estado del AFD es un subconjunto de estados del AFN.

Convertir un AFN a un AFD (Algoritmo Construcción de conjuntos)

Operaciones para localizar los conjuntos de los estados del AFN (s representa un estado del AFN, y T un conjunto de estados del AFN).

Operación	Descripción
cerradura- ϵ (s)	Conjunto de estados del AFN alcanzables desde el edo. s del AFN con transiciones ϵ solamente.
cerradura- ϵ (T)	Conjunto de estados del AFN alcanzables desde algún edo. s en T con transiciones ϵ solamente.
mueve(T, a)	Conjunto de estados del AFN hacia los cuales hay una transición con el símbolo de entrada a desde algún edo. s en T del AFN.

Convertir un AFN a un AFD (A. Construcción de conjuntos)

Algoritmo principal de construcción de conjuntos

Al inicio, **cerradura- ϵ (S_0)** es el único estado dentro de estadosD y no está marcado

while haya un edo no marcado T en estadosD

 marcar T

 for cada símbolo de entrada ***a***

 U:= cerradura- ϵ (mueve(T,a));

 if U no está en estadosD

 Añadir U como estado no marcado en estadosD

 endif

 tranD[T,a]:=U

 endFor

endWhile

Algoritmo para el calculo de la CERRADURA- ϵ

Meter todos los estados de T en pila

Inicializar cerradura- ϵ (T) con T

while pila no esté vacía

 sacar t, el elemento tope de la pila

 for cada transición ϵ de t a v

 if v no está en cerradura- ϵ (T)

 añadir v a cerradura- ϵ (T)

 meter v en la pila

 endif

 endFor

endWhile

Algoritmo mueve(T, a)

conjuntoMueve={}

for cada estado t de T

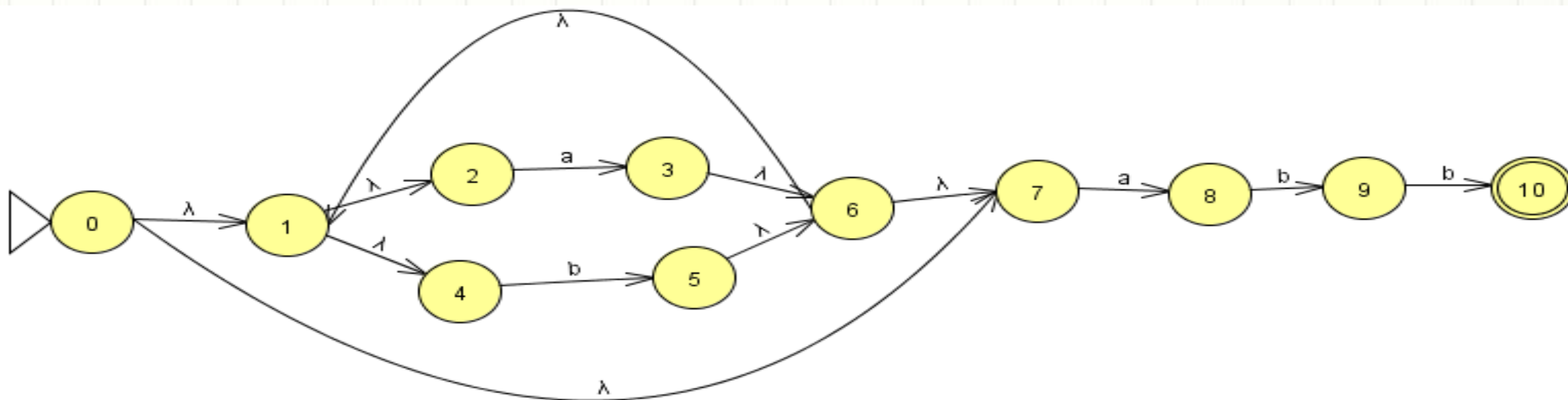
if hay una transición a de t a v y v no está en conjuntoMueve

 añadir v a conjuntoMueve

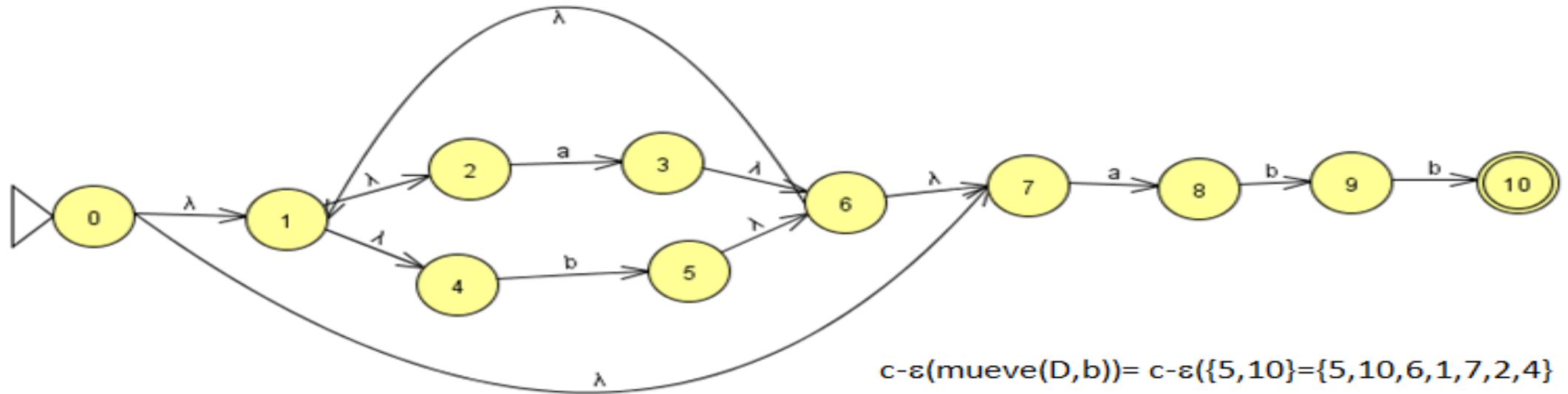
endif

endFor

Dado el siguiente AFN que representa la expresión regular $(a|b)^*abb$.
Obtenga su correspondiente AFD aplicando el algoritmo de construcción de conjuntos.



2. Dado el Autómata Finito No Determinista (AFN) que reconoce la expresión regular $(a|b)^*abb$, construya su respectivo AFD mediante el algoritmo de construcción de conjuntos.



$$c-\varepsilon(0) = \{0,1,7,2,4\}$$

$$A=\{0,1,2,4,7\}$$

$$c-\varepsilon(\text{mueva}(A,a))=c-\varepsilon(\{3,8\})=\{3,8,6,1,7,2,4\}$$

$$B=\{1,2,3,4,6,7,8\}$$

$$c-\varepsilon(\text{mueva}(A,b))=c-\varepsilon(\{5\})=\{5,6,1,7,2,4\}$$

$$C=\{1,2,4,5,6,7\}$$

$$c-\varepsilon(\text{mueva}(B,a))=c-\varepsilon(\{3,8\})=B$$

$$c-\varepsilon(\text{mueva}(B,b))=c-\varepsilon(\{5,9\})=\{5,9,6,1,7,2,4\}$$

$$D=\{1,2,4,5,6,7,9\}$$

$$c-\varepsilon(\text{mueva}(C,a))=c-\varepsilon(\{3,8\})=B$$

$$c-\varepsilon(\text{mueva}(C,b))=c-\varepsilon(\{5\})=C$$

$$c-\varepsilon(\text{mueva}(D,a))=c-\varepsilon(\{3,8\})=B$$

$$c-\varepsilon(\text{mueva}(D,b))=c-\varepsilon(\{5,10\})=\{5,10,6,1,7,2,4\}$$

$$E=\{1,2,4,5,6,7,10\}$$

$$c-\varepsilon(\text{mueva}(E,a))=c-\varepsilon(\{3,8\})=B$$

$$c-\varepsilon(\text{mueva}(E,b))=c-\varepsilon(\{5\})=C$$

$$s_0=A$$

$$S=\{A,B,C,D,E\}$$

$$F=\{E\}$$

$$\Sigma=a,b$$

Tabla de transiciones del AFD

Edo	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

AFD obtenido del AFN anterior correspondiente a la expresión regular $(a|b)^*abb$

$s_0 = A$

$S = \{A, B, C, D, E\}$

$F = \{E\}$

$\Sigma = a, b$

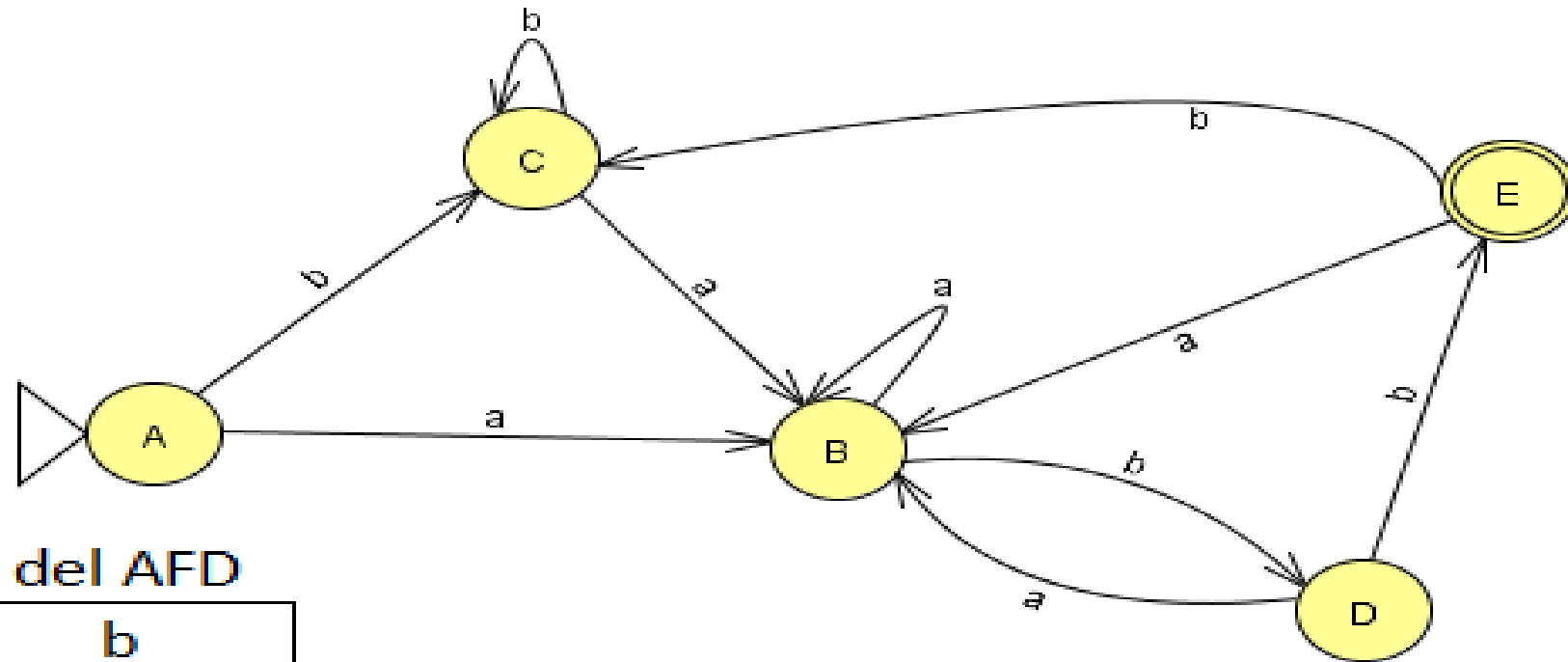
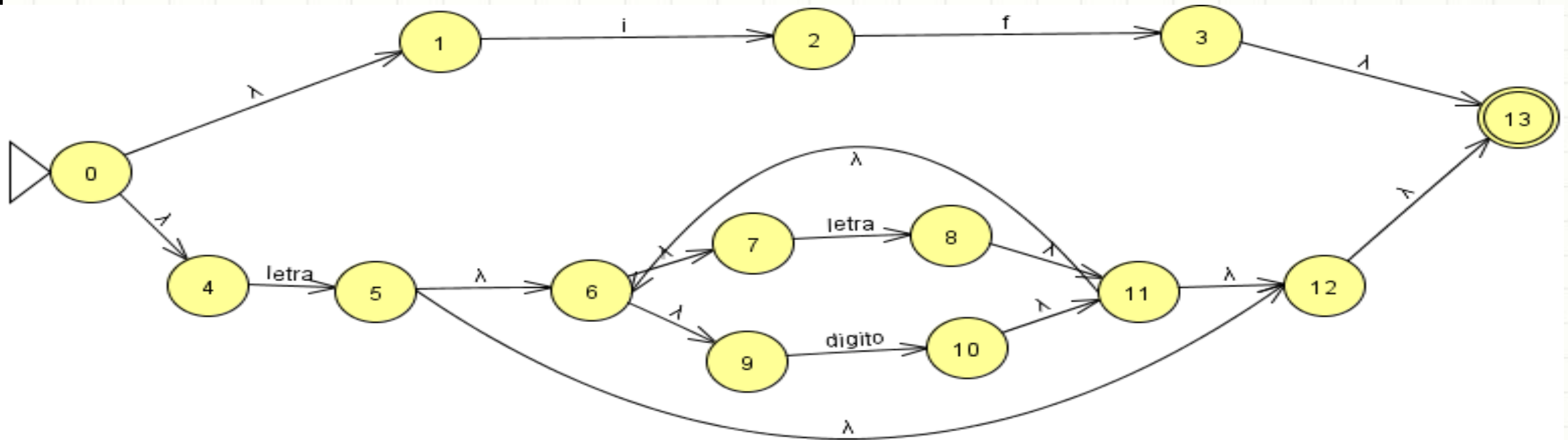


Tabla de transiciones del AFD

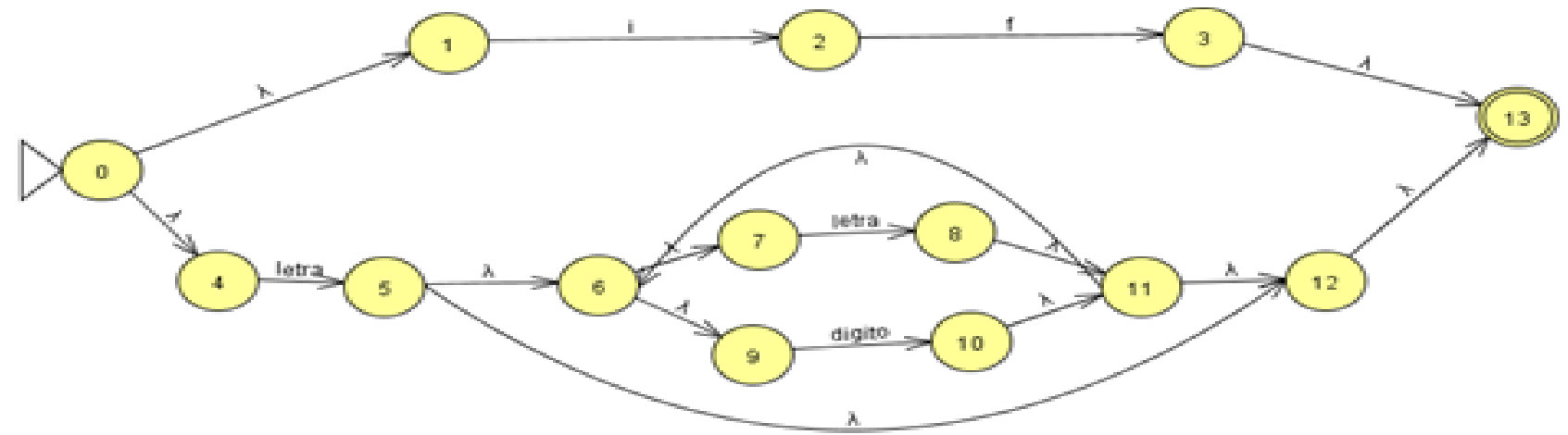
Edo	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

Convertir un AFN a un AFD (Algoritmo Construcción de conjuntos)

Dado el siguiente AFN que reconoce la expresión regular $if|letra(letra|dígito)^*$ obtenga su AFD aplicando el algoritmo de construcción de conjuntos.



2. Dado el AFN que reconoce la expresión regular $i f \mid letra(letra \mid digito)^*$, construya su respectivo AFD mediante el algoritmo de construcción de conjuntos.



$c-\epsilon(0) = \{0,1,4\}$

$A = \{0,1,4\}$

$c-\epsilon(\text{mueva}(A,i)) = c-\epsilon(\{2,5\}) = \{2,5,6,12,7,9,13\}$

$B = \{2,5,6,7,9,12,13\}$

$c-\epsilon(\text{mueva}(A,letra-i)) = c-\epsilon(\{5\}) = \{5,6,12,7,9,13\}$

$C = \{5,6,7,9,12,13\}$

$c-\epsilon(\text{mueva}(B,f)) = c-\epsilon(\{3,8\}) = \{3,8,13,11,6,12,7,9\}$

$D = \{3,6,7,8,9,11,12,13\}$

$c-\epsilon(\text{mueva}(B,letra-f)) = c-\epsilon(\{8\}) = \{8,11,6,12,7,9,13\}$

$E = \{6,7,8,9,11,12,13\}$

$c-\epsilon(\text{mueva}(B,digito)) = c-\epsilon(\{10\}) = \{10,11,6,12,7,9,13\}$

$F = \{6,7,9,10,11,12,13\}$

$c-\epsilon(\text{mueva}(C,letra)) = c-\epsilon(\{8\}) = E$

$c-\epsilon(\text{mueva}(C,digito)) = c-\epsilon(\{10\}) = F$

$c-\epsilon(\text{mueva}(D,letra)) = c-\epsilon(\{8\}) = E$

$c-\epsilon(\text{mueva}(D,digito)) = c-\epsilon(\{10\}) = F$

$c-\epsilon(\text{mueva}(E,letra)) = c-\epsilon(\{8\}) = E$

$c-\epsilon(\text{mueva}(E,digito)) = c-\epsilon(\{10\}) = F$

$c-\epsilon(\text{mueva}(F,letra)) = c-\epsilon(\{8\}) = E$

$c-\epsilon(\text{mueva}(F,digito)) = c-\epsilon(\{10\}) = F$

$s_0 = A$

$S = \{A,B,C,D,E,F\}$

$F = \{B,C,D,E,F\}$

$\Sigma = i, f, letra-i, letra-f, letra, digito$

Tabla de transiciones del AFD

Edo	i	letra-i	f	letra-f	letra	digito
A	B	C				
B			D	E		F
C					E	F
D					E	F
E					E	F
F					E	F

AFD obtenido del AFN de la diapositiva anterior, correspondiente a la expresión regular :

$if \mid letra(letra \mid digito)^*$

$S_0 = A$

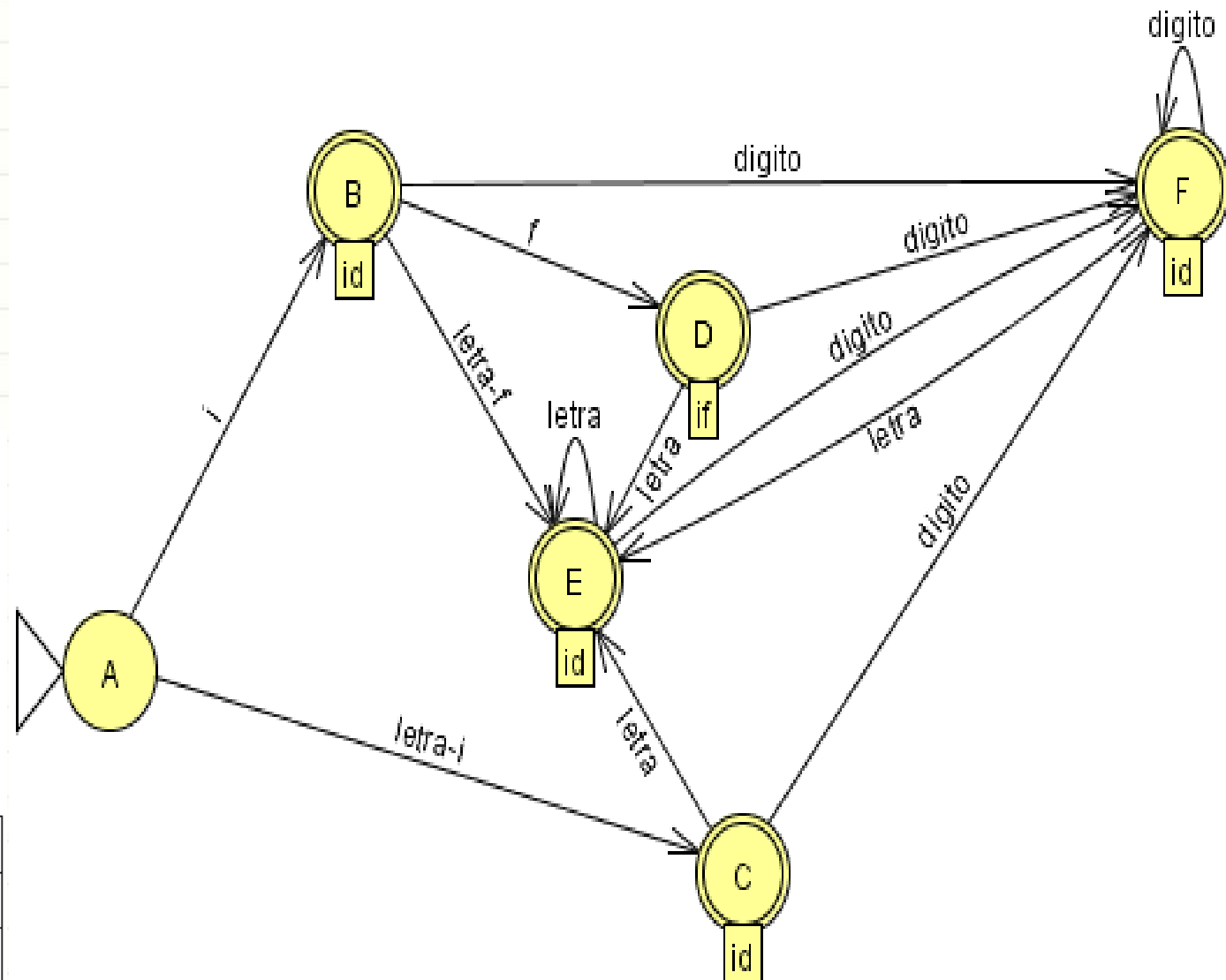
$S = \{A, B, C, D, E, F\}$

$F = \{B, C, D, E, F\}$

$\Sigma = \{i, f, letra-i, letra-f, letra, digito\}$

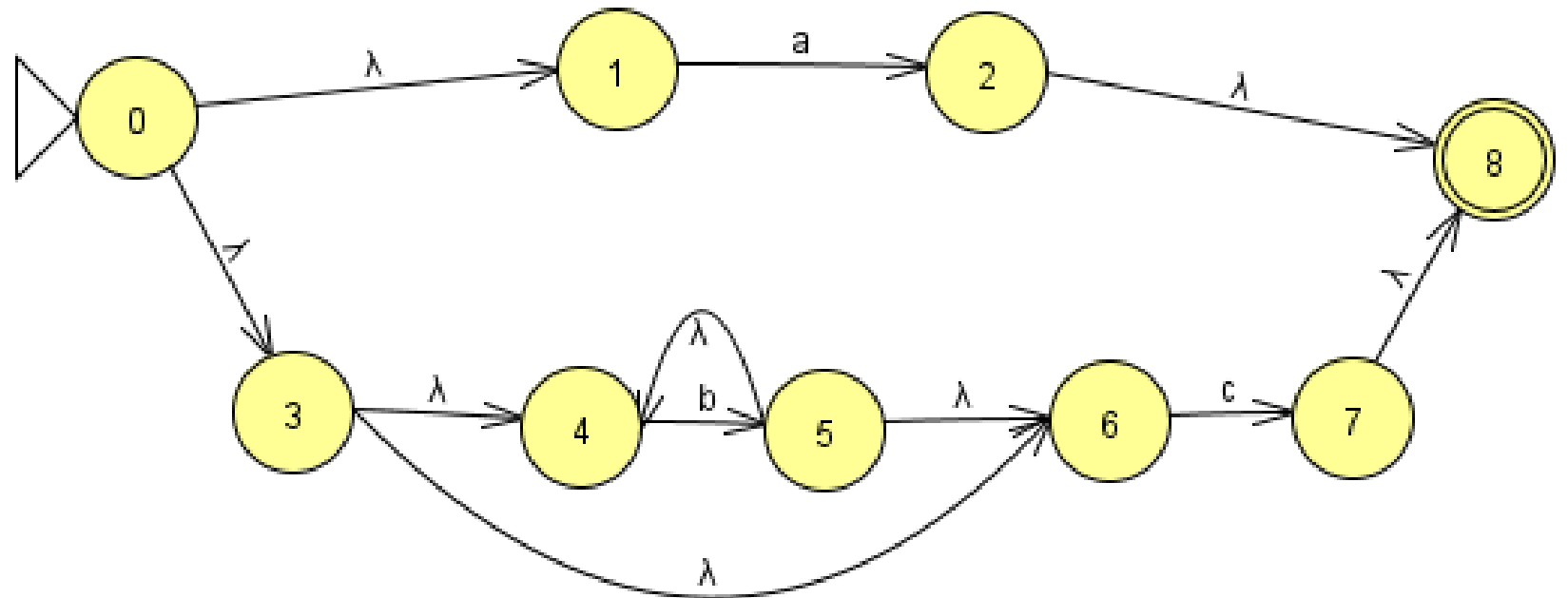
Tabla de transiciones del AFD

Edo	i	letra-i	f	letra-f	letra	digito
A	B	C				
B			D	E		F
C					E	F
D					E	F
E					E	F
F					E	F



Convertir un AFN a un AFD (Algoritmo Construcción de conjuntos)

Dado el siguiente AFN que representa la expresión regular $a|b^*c$, obtenga su correspondiente AFD aplicando el algoritmo de construcción de conjuntos.



2. Especificación y reconocimiento de los símbolos de un lenguaje

- Ejemplos de expresiones regulares para un lenguaje de programación

letra → a-zA-Z

digito → 0|1|2|3|4|5|6|7|8|9

caracterEspecial → +|-|*|/|=|;|{|}|[|]|_| etc

id → letra(letra|digito)*

nint → (digito)+

nfloat → (digito)+.(digito)+

literalCad → "(letra|digito|caracterEspecial)*"

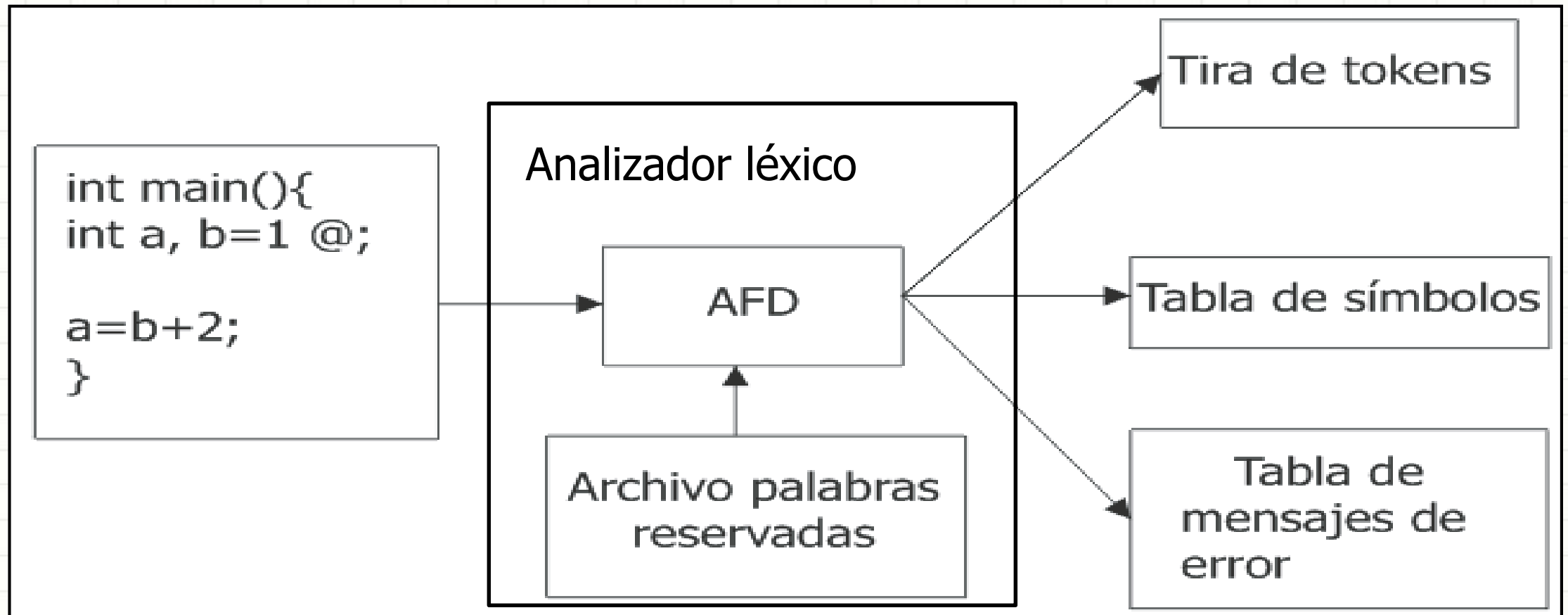
(→ (

) →)

...



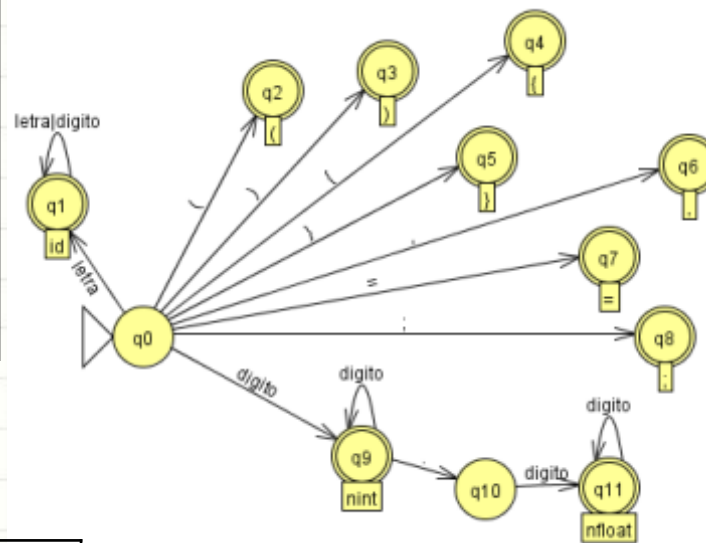
3. Diseño e implementación de un analizador léxico.



Programa fuente

```
int main() {  
int a, b=1 @;  
  
a=b+2;  
}
```

AFD



1. Tira de tokens

# línea	Lexema	Token
1	int	int
1	main	main
1	(	(
1	)	)
1	{	{
2	int	int
2	a	id."a"
2	,	,
2	b	id."b"
2	=	=
2	1	nint.1
2	;	;
4	a	id."a"
4	=	=
4	b	id."a"
4	+	+
4	2	nint.2
4	;	;
5	}	}

2. Tabla de símbolos

id	valor	función
a		main
b		main

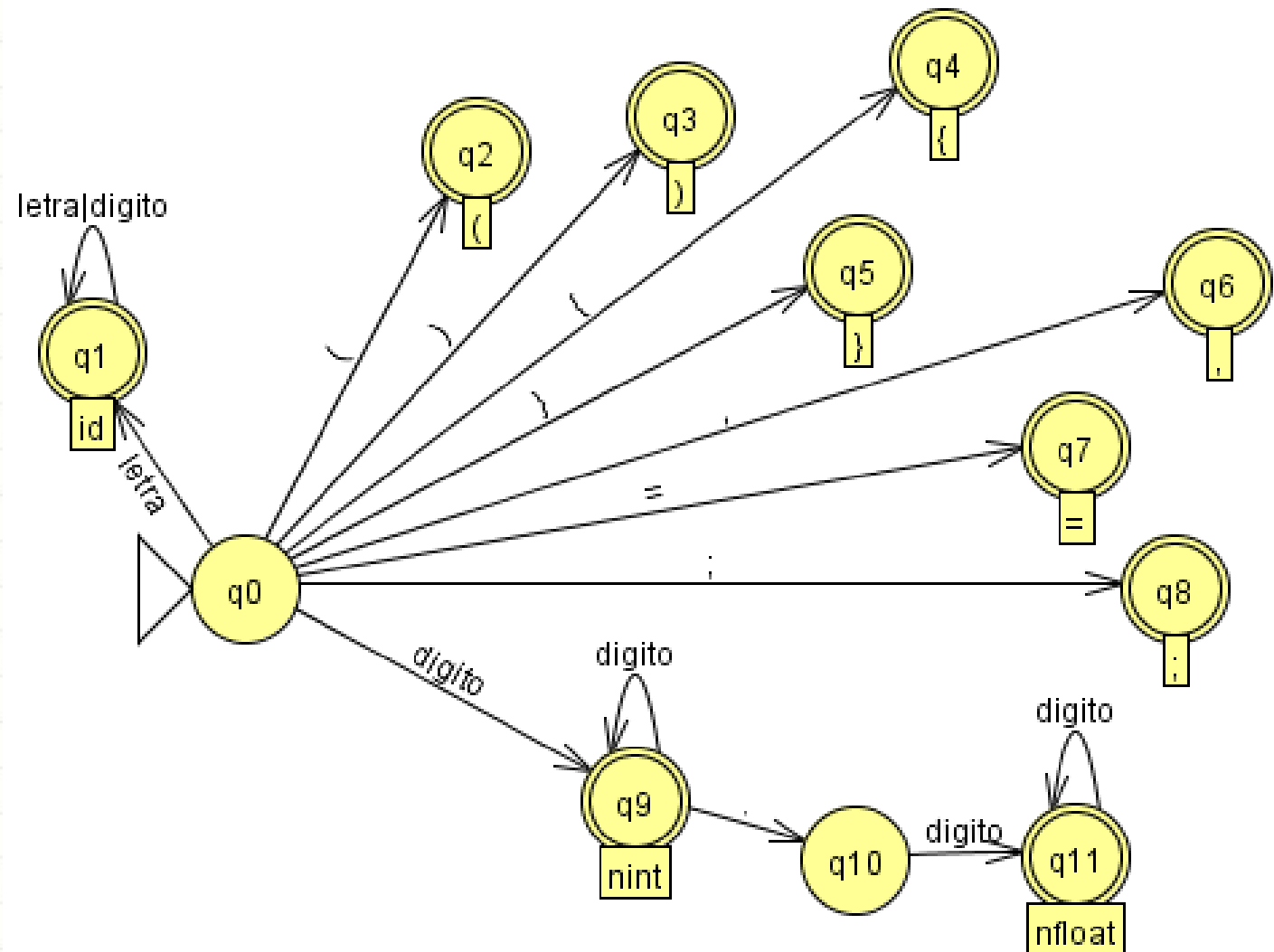
3. Tabla de errores

# línea	Descripción
	En función 'main'
2	Error: @ símbolo no definido.

3. Diseño e implementación de un analizador léxico.

Pequeño conjunto de expresiones regulares para el lenguaje C, que permitirán crear un AFD a través de los algoritmos vistos anteriormente o de manera directa, para la implementación de un analizador léxico.

$\text{letra}(\text{letra} | \text{digito})^* | (|) | \{ | \}$
 $| , | = | ; |$
 $\text{digito}^+ | \text{digito}^+.\text{digito}^+$



Algoritmo de simulación muy simple de un AFD [1]

```
s = s0;  
c = sigCar();  
while ( c != eof ) {  
    s = mover(s, c);  
    c = sigCar();  
}  
if ( s está en F ) return "si";  
else return "no";
```

Figura 3.27: Simulación de un AFD

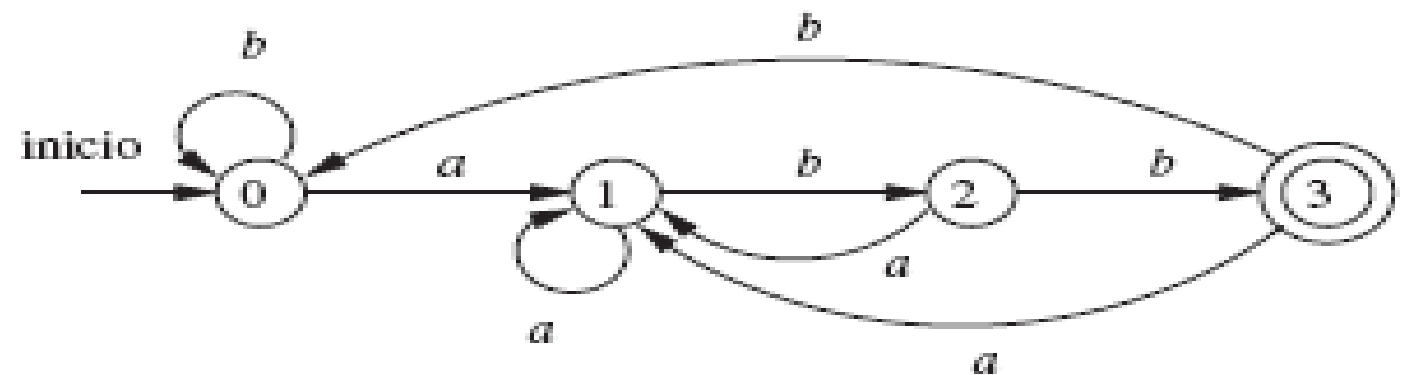


Figura 3.28: AFD que acepta a $(a|b)^*abb$

3. Diseño e implementación de un analizador léxico.

Algunas consideraciones para el analizador léxico del proyecto final.

- Seleccionar el lenguaje origen, se recomienda que sea de alto nivel e identifique si es estructurado ú orientado a objetos.
- En el tercer parcial se realizará el análisis semántico, por lo cual, tendrá que seleccionar el lenguaje objeto. El lenguaje objeto debe ser consecuente al lenguaje fuente, es decir, Estructurado – Estructurado; u Orientado a Objetos- Orientado a Objetos. Ejemplo C a pascal, Java a C++, etc.
- **Ver documento de especificación de requerimientos.**

Sugerencia de algunas clases para la implementación del analizador léxico

```
public class EstadosFinales {  
    private int edoFinal;  
    private String aceptacion;
```

```
public class TokenALexico {  
    private int linea;  
    private String token;  
    private String lexema;  
    private String valorLexico;
```

```
public class AFDAlexico {  
    private HashMap<Integer, String> estadosFinales = new HashMap<>();  
    public AFDAlexico(){  
        estadosFinales.put(1, "id");  
        estadosFinales.put(2, "(");  
        estadosFinales.put(3, ")");  
        estadosFinales.put(4, "{");  
        estadosFinales.put(5, "}");
```

```
1 package compi.presentacion.alexico;
```

```
2
```

```
3 import java.io.IOException;
```

```
14
```

```
15 public class AnalisisLexico {
```

```
16     ArrayList<TokenALexico> tiraTokens = new ArrayList<TokenALexico>();
```

```
17     ArrayList<Errores> TablaErrores = new ArrayList<Errores>();
```

```
18     ArrayList<Simbolo> TablaSimbolos = new ArrayList<Simbolo>();
```

```
while (i < lineaCod.size()) {
```

```
    copia = lineaCod.get(i);
```

```
    j = 0;
```

```
    cont = 0;
```

```
    edoact = 0;
```

```
    while (j < copia.length() && copia.length() != 0) {
```

```
        caracter = copia.charAt(j);
```

```
        if (edoact == 0 && ((caracter >= 'a' && caracter <= 'z') || (caracter >= 'A' && caracter <= 'Z'))) {
```

```
            edoact = 1;
```

```
            auxiliar[cont] = caracter;
```

```
            cont++;
```

```
            j++;
```

```
        } else if (edoact == 1 && ((caracter >= 'a' && caracter <= 'z') || (caracter >= 'A' && caracter <= 'Z'))
```

```
            || (caracter >= '0' && caracter <= '9')) {
```

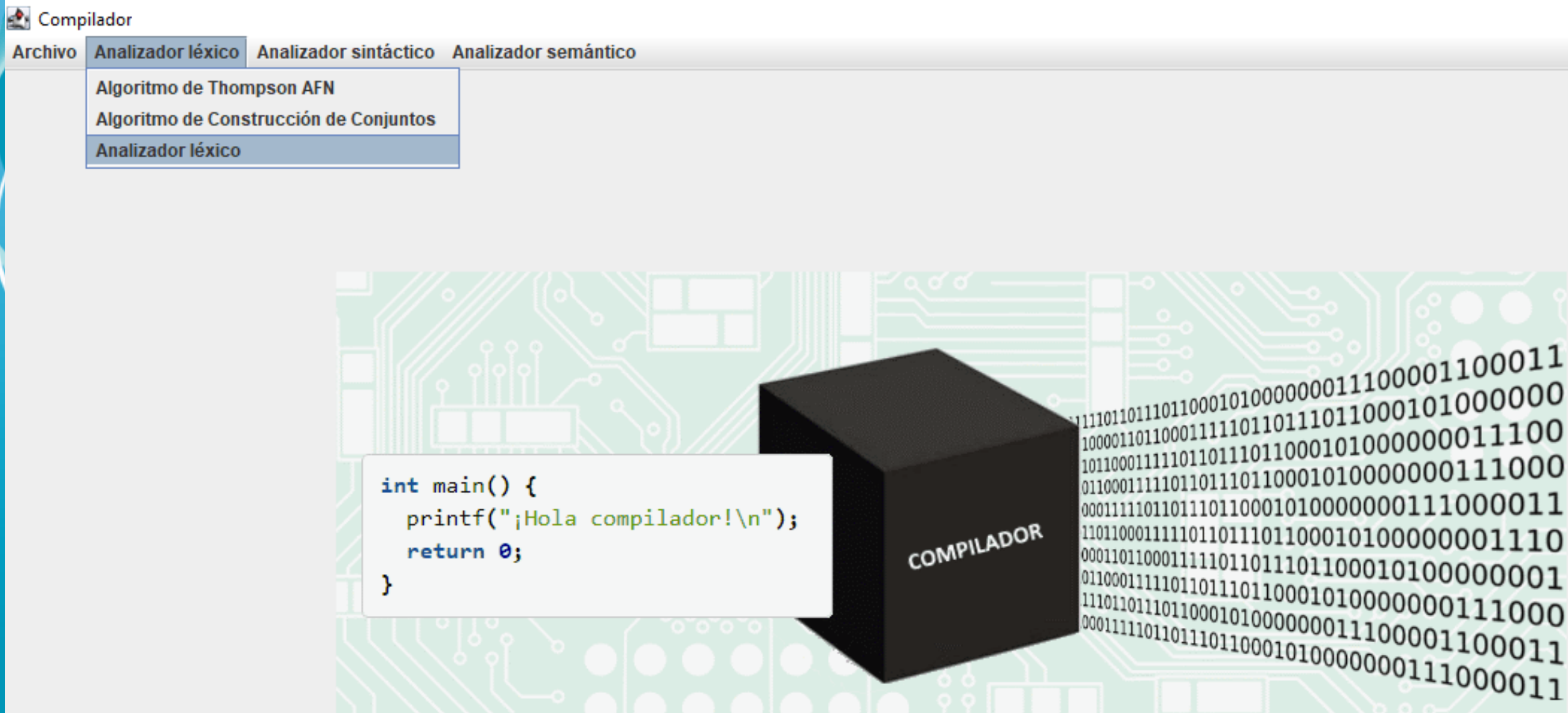
```
            edoact = 1;
```

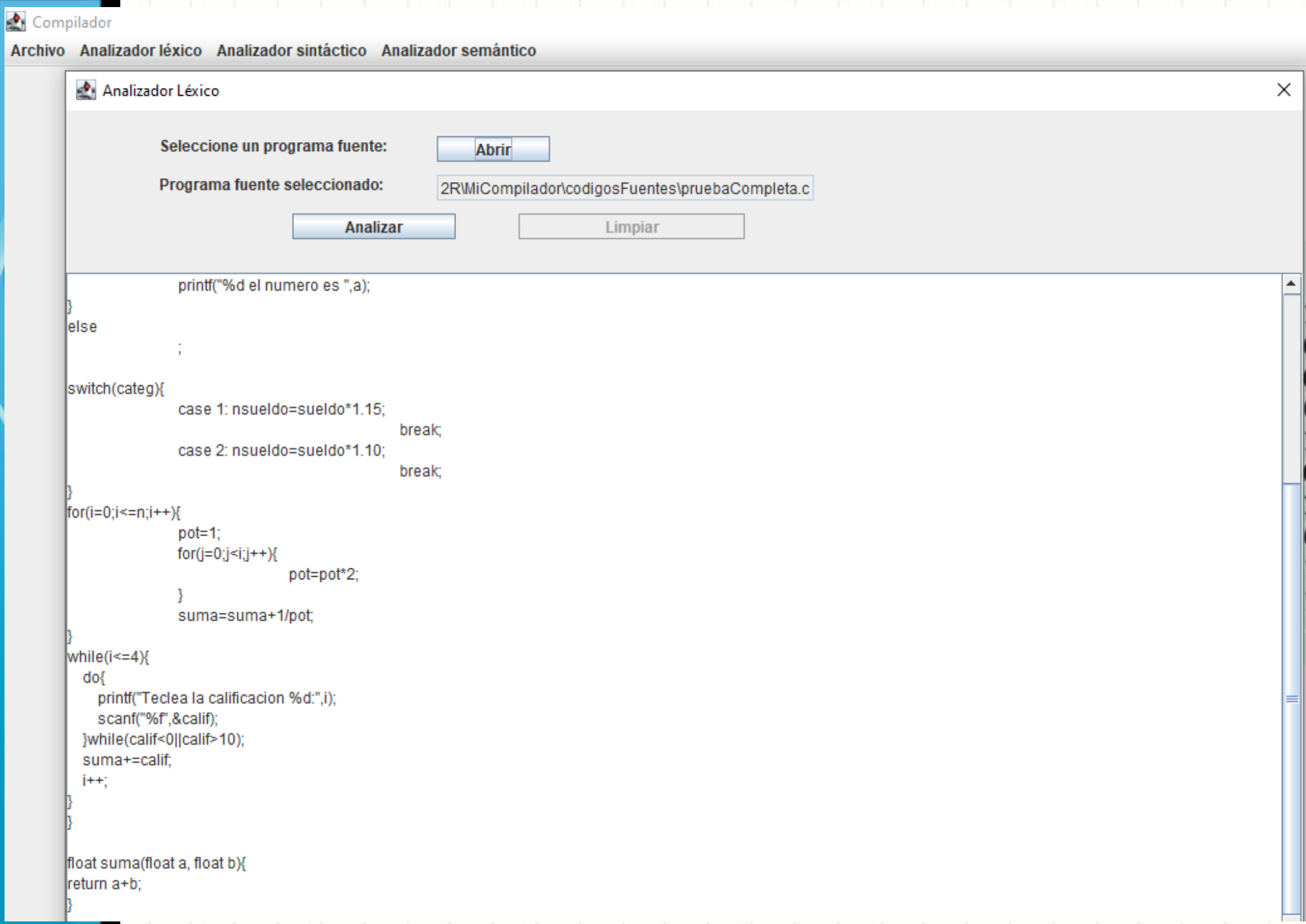
```
            auxiliar[cont] = caracter;
```

```
            cont++;
```

```
            j++;
```

3. Sugerencia del diseño e implementación de un analizador léxico: Interfaz principal





**Programa
fuente a
compilar**

Seleccione un programa fuente:

Abrir

Programa fuente seleccionado:

2RMiCompilador\codigosFuentes\pruebaCompleta.c

Analiza

Listado de la tira de tokens

```
printf("%d el numero es ",a);  
}  
else  
;  
switch(categ){  
case 1: nsueldo=sueldo*1.15;  
case 2: nsueldo=sueldo*1.10;  
}  
for(i=0;i<=n;i++){  
    pot=1;  
    for(j=0;j<i;j++){  
        pot=pot*2;  
    }  
    suma=suma+1/pot;  
}  
while(i<=4){  
    do{  
        printf("Teclea la calificacion %d:",i);  
        scanf("%f",&calif);  
    }while(calif<0||calif>10);  
    suma+=calif;  
    i++;  
}  
}  
float suma(float a, float b){  
return a+b;  
}
```

#linea	Lexema	Token
1	int	int
1	main	main
1	(	(
1	)	)
1	{	{
4	int	int
4	a	id.a
4	,	,
4	b	id.b
4	;	;
8	int	int
8	*	*
8	aptr	id.aptr
8	,	,
8	vec	id.vec
8	[	[
8	10	nint.10
8	]	]
8	,	,
8	categ	id.categ
8	=	=
8	2	nint.2
8	;	;
9	int	int
9	*	*
9	aVecPtr	id.aVecPtr
9	[	[
9	3	nint.3
9	]	]
9	;	;
10	float	float
10	vecFloat	id.vecFloat

1. Tira
de
tokens
(compon
entes
léxicos)

Compilador

Archivo Analizador léxico Analizador sintáctico Analizador semántico

Analizador Léxico

Seleccione un programa fuente:

Programa fuente seleccionado: 2R\MiCompilador\codigosFuentes\pruebaCompleta.c

```
printf("%d el numero es ",a);
}
else
;
switch(categ){
case 1: nsueldo=sueldo*1.15; break;
case 2: nsueldo=sueldo*1.10; break;
}
for(i=0;i<=n;i++){
pot=1;
for(j=0;j<i;j++){
pot=pot*2;
}
suma=suma+1/pot;
}
while(i<=4){
do{
printf("Teclea la calificacion %d:",i);
scanf("%f",&calif);
}while(calif<0||calif>10);
suma+=calif;
i++;
}
}

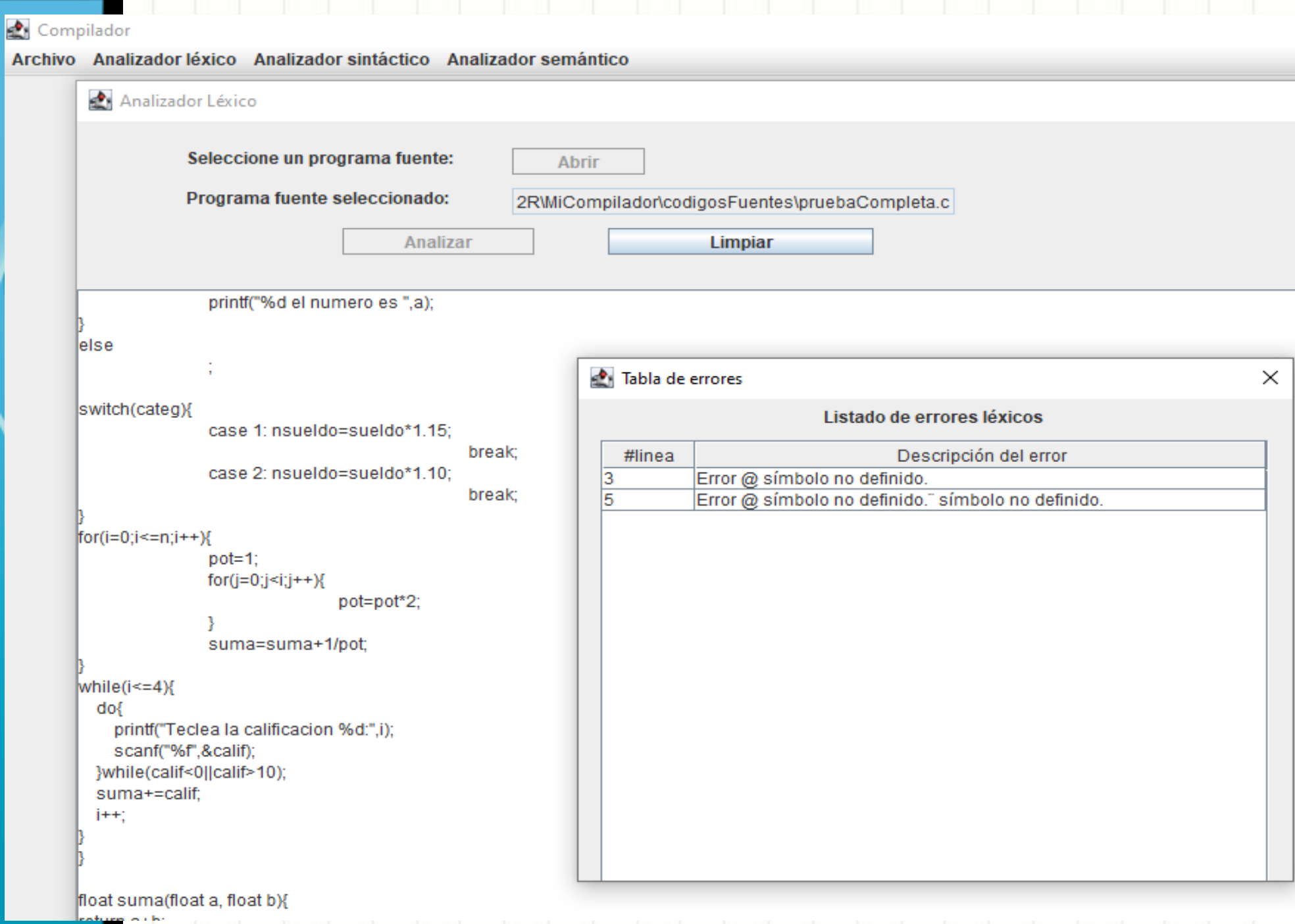
float suma(float a, float b){
return a+b;
}
```

Tabla de símbolos

Listado de la tabla de símbolos o identificadores

#Id	Nombre del identificador
1	a
2	b
3	aptr
4	vec
5	categ
6	aVecPtr
7	vecFloat
8	sueldo
9	nsueldo
10	i
11	n
12	pot
13	j
14	suma
15	calif

2. Tabla de símbolos



3. Tabla de errores

4. Control de errores léxicos

- Pocos errores se pueden detectar a nivel del léxico
 - $fi(a == f(x))$ ¿Error léxico?
- Recuperación de errores en modo de pánico.
 - Se borran o descartan los caracteres sucesivos de la entrada restante hasta que el analizador léxico pueda encontrar un tóken válido.

4. Control de errores léxicos

Otras posibles acciones de recuperación de errores:

- Borrar o descartar un carácter extraño
- Insertar un carácter que faltaba
- Reemplazar un carácter incorrecto por otro correcto
- Intercambiar dos caracteres adyacentes.

5. Generadores de analizadores léxicos

Instalar la herramienta Flex, para el desarrollo de analizadores léxicos.

- flex-2.5.4a-1

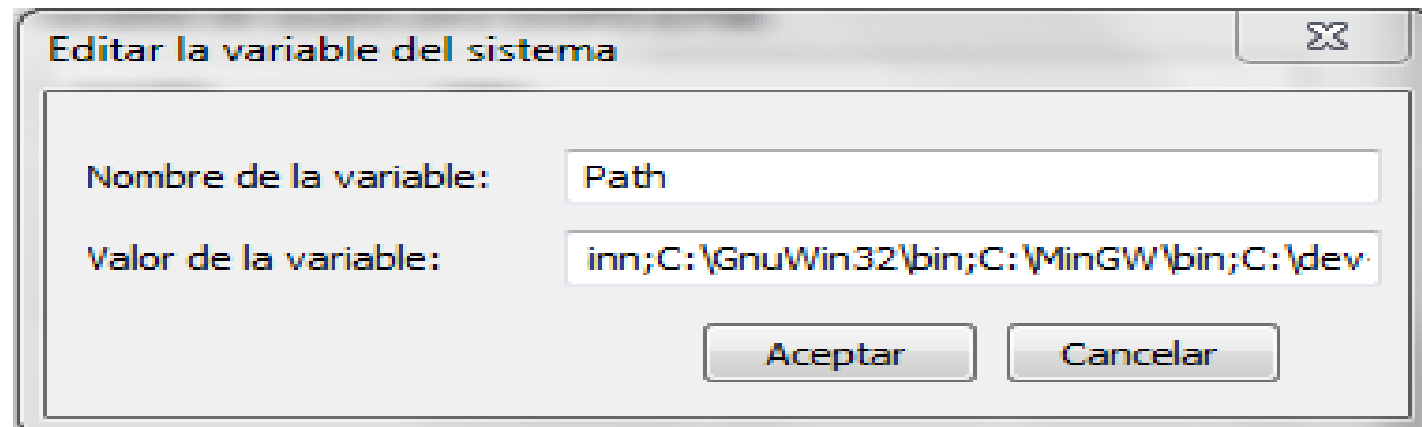
<http://gnuwin32.sourceforge.net/packages/flex.htm>

Opción "Complete package, except sources"

- Recomendable instalar en C:\GnuWin32

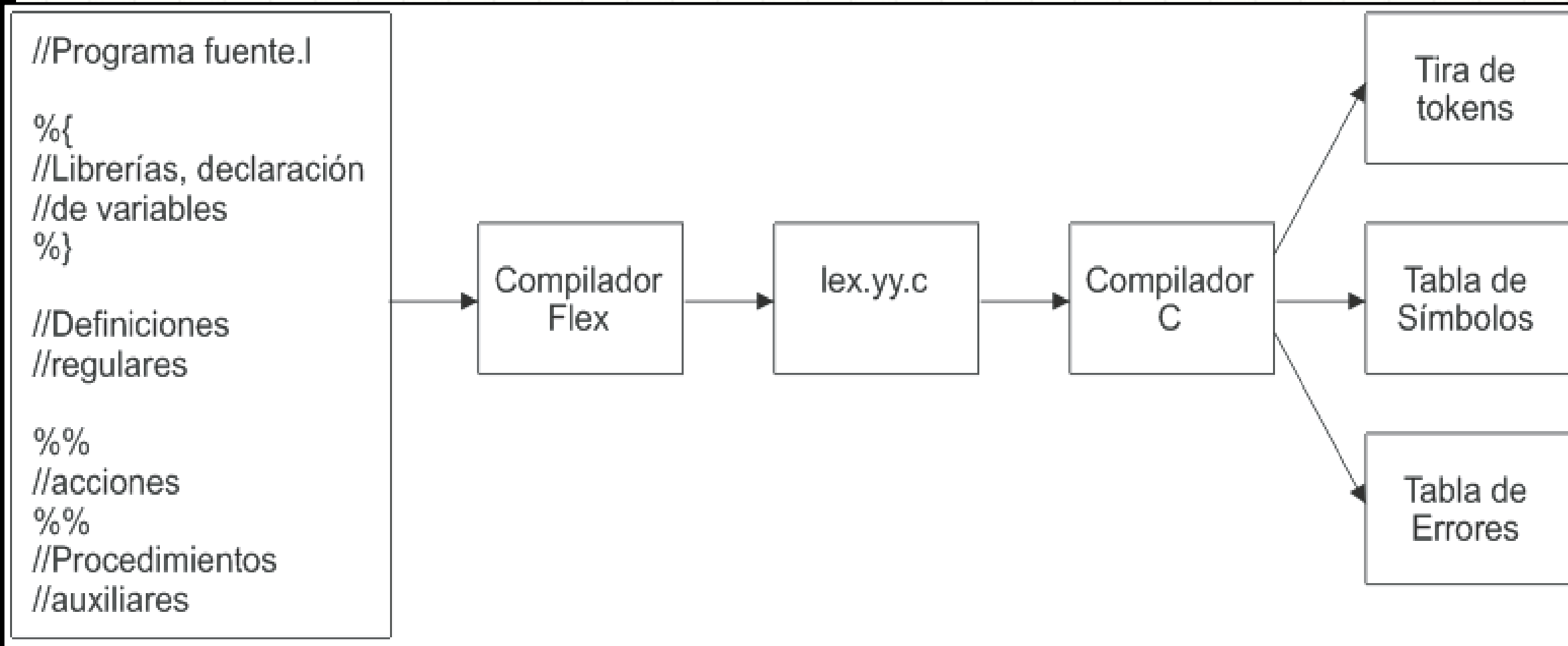
Agregar en la variable Path del sistema, la ruta donde se instaló Flex, en este caso C:\GnuWin32

Así mismo, en caso de tener instalado un compilador de C, ejemplo Dev-C, agregar también la ruta donde se encuentra instalado en la misma variable Path. De lo contrario, instalar un compilador de C.



5. Generadores de analizadores léxicos.

Flujo del analizador léxico



```

/* Esto indica a Flex que lea solo un archivo de entrada */
%option noyywrap
%{
/* Código en C que será copiado */
#include <stdio.h>
}%
//En este ejemplo no hay definición de expresiones regurales
%%
    /*** Sección de reglas y acciones ***/
/* [0-9]+ identifica una cadena de uno o más dígitos */
[0-9]+ {
    /* yytext es una cadena que contiene el texto coincidente. */
    printf("Encontrado un entero: %s\n", yytext);
}
. { /* Ignora todos los demás caracteres. */ }
%%
/*** Sección de código en C ***/
int main(void){
    /* Ejecuta el "lexico", y después termina. */
    yylex();
    return 0;
}

```

Explicación de las partes de un programa fuente en Flex.

```
%option noyywrap
```

```
%{
```

```
/*
```

```
Ejemplo de entrada al generador Flex
```

```
*/
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
typedef struct tabla{
```

```
    char nombre[20];
```

```
    struct tabla * sig;
```

```
}tablaSimb;
```

```
tablaSimb *tSimb,*nuevo,*temp;
```

```
FILE *tiraTokens=NULL, *errores=NULL;
```

```
int numLinea=1;
```

```
void escribirTiraTokens(char token[]);
```

```
void escribirError();
```

```
void insertar(tablaSimb **tSimb,tablaSimb *nuevo);
```

```
tablaSimb * buscar(tablaSimb *tSimb,char nombre[]);
```

```
void imprimir(tablaSimb *tSimb);
```

```
%}
```

Ejemplo de un programa fuente de nombre **lexico.l**, que permitirá crear el analizador léxico para las expresiones regulares que en el se definen.

Se genera la tira de tokensa, la tabla de símbolos y la tabla de errores.

letra	[a-zA-Z]
digito	[0-9]
identificador	{letra} ({letra} {digito}) *
nint	{digito} +
nfloat	{digito} + (\. {digito} +) ?
literal	\ " (\\ \. [^ \\ "]) * \ "

```

%%
[\\n]          ++numLinea;
"float"        {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"float");
                escribirTiraTokens("float");}
"int"          {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"int");
                escribirTiraTokens("int");}
"main"         {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"main");
                escribirTiraTokens("main");}
"printf"       {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"printf");
                escribirTiraTokens("printf");}
"return"       {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"return");
                escribirTiraTokens("return");}
"("            {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"(");
                escribirTiraTokens("(");}
")"           {printf("%-10d%-20s%-10s\\n",numLinea, yytext,")");
                escribirTiraTokens(")");}
"{"           {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"{");
                escribirTiraTokens("{");}
"}"           {printf("%-10d%-20s%-10s\\n",numLinea, yytext,"}");
                escribirTiraTokens("}");}
{literal}      {printf("%-15d%-20s%-10s\\n",numLinea,yytext,"literal");
                escribirTiraTokens("literal");}

```


{identificador}	<pre> {printf("%-10d%-20s%-10s\n",numLinea, yytext,"id"); escribirTiraTokens("id"); temp = buscar(tSimb,yytext); if (temp == NULL){ nuevo=(tablaSimb *) malloc(sizeof(tablaSimb)); if(nuevo!=NULL){ strcpy(nuevo->nombre, yytext); insertar(&tSimb, nuevo); } } } </pre>
{nint}	<pre> {printf("%-10d%-20s%-10s\n",numLinea,yytext,"nint"); escribirTiraTokens("nint");} </pre>
{nfloat}	<pre> {printf("%-10d%-20s%-10s\n",numLinea,yytext,"nfloat"); escribirTiraTokens("nfloat");} </pre>
[,]	<pre> {printf("%-10d%-20s%-10s\n",numLinea,yytext,","); escribirTiraTokens(",");} </pre>
[;]	<pre> {printf("%-10d%-20s%-10s\n",numLinea,yytext,";"); escribirTiraTokens(";");} </pre>
[\t\v\f]	<pre> {/*ignorar*/} </pre>
.	<pre> {escribirError();} </pre>

%%

```
main(int argc, char *argv[]){
yyin = fopen("prueba2.c","r");
tiraTokens=fopen("tiraTokens.txt","w");
errores=fopen("errores.txt","w");
tSimb=NULL;
if(yyin!=NULL){
    printf("%-10s%-20s%-10s\n","No linea","lexema","token");
    fprintf(tiraTokens,"%-10s%-20s%-10s\n","No linea","lexema","token");
    fprintf(errores,"%-10s%-20s\n","No linea","Descripcion");
    yylex();
    imprimir(tSimb);
}

void escribirTiraTokens(char token[]){
fprintf(tiraTokens,"%-10d%-20s%-10s\n",numLinea, yytext,token);
}

void escribirError(){
fprintf(errores,"%-10d:%s%s%s\n",numLinea,"Error lexico, simbolo",
yytext," no definido");
}

void insertar(tablaSimb **tSimb,tablaSimb *nuevo){
    nuevo->sig = *tSimb;
    *tSimb = nuevo;
}
```

```
tablaSimb * buscar(tablaSimb *tSimb,char nombre[]){  
while ( tSimb != NULL && strcmp(nombre, tSimb->nombre) )  
    tSimb = tSimb->sig;  
return tSimb;  
}  
  
void imprimir(tablaSimb *tSimb){  
printf("\n\nTabla de simbolos\n");  
while (tSimb != NULL){  
    printf("%s\n", tSimb->nombre);  
    tSimb = tSimb->sig;  
}  
}
```

- Instrucciones de compilación:

flex lexico.l

gcc -o lexico lex.yy.c

- Instrucciones de ejecución:

lexico

5. Generadores de analizadores léxicos.

prueba2.c

```

1 int main() {
2     //comentario de linea
3     int a,b;
4     printf("hola \n");
5     a=b;
6     return 0;
7 }
```

```

D:\misProgFlexBison\analizadorLexicoyTablaSimbolos>lexico
No linea  lexema      token
1          int        int
1          main       main
1          (          (
1          )          )
1          {          {
2          comentario id
2          de         id
2          linea      id
3          int        int
3          a          id
3          ,          ,
3          b          id
3          ;          ;
4          printf     printf
4          (          (
4              "hola \n"      literal
4          )          )
4          ;          ;
5          a          id
5          b          id
5          ;          ;
6          return     return
6          0          nint
6          ;          ;
7          }          }
```

5. Generadores de analizadores léxicos.

```
prueba2.c x
1 int main() {
2     //comentario de linea
3     int a,b;
4     printf("hola \n");
5     a=b;
6     return 0;
7 }
```

Tabla de simbolos

b

a

linea

de

comentario

Tabla de errores

errores.txt x

1	No linea	Descripcion
2	2	:Error lexico, simbolo/ no definido
3	2	:Error lexico, simbolo/ no definido
4	5	:Error lexico, simbolo= no definido

Referencias

1. Compiladores. Principios, Técnicas y Herramientas (2a ed.). Aho, A.V. Pearson Educación. 2008.
2. Construcción de Compiladores. Principios y práctica. Louden, K. C. Thomson Editores. 2005
3. Traductores, Compiladores e Interpretes. Aguilera, S. M & Gálvez, R. S.